Middleware Architecture (IS3108/SCS3203)

Scenario-Based Class Assessment - 2025

Duration - 2 Hours

Scenario - SriToysRus Ltd

SriToysRus Ltd (STRU) is a Sri Lankan-based company that specialises in promoting toys to children below 12 years of age. The company has 15 branches across the island and sells toys that are either imported or manufactured locally. Due to the high demand for sales in Colombo, the company has already planned to open several new branches in a few major cities. The management of the company is interested in extending the reach of the business using Internet technology. They are interested in exposing the overall inventory to the public using an Internet-based application. Some of the key functionalities expected out of this system are listed below.

- Users should be able to search for toys available for sale through the system.
- Detailed descriptions of the selected toys, such as category, brand, country of origin, price, discounts, age limits, educational aspects, etc, should be displayed.
- The users should be able to create personalised accounts to register and log in to the system.
- The personalised account/profile will assist in having a personalised view/history of the system - E.g. past purchases, wish list, etc. Users should be able to register their credit cards and make purchases online.
- The purchases made online through the STRU platform will be delivered to the users' registered addresses. The delivery of the purchased goods to the users will be done by an existing delivery company that already has a platform to track and monitor the overall distribution process.

A centralised inventory control system should be maintained so that the sale of products, either through the online platform or the outlets, reflects the correct view of available stock. A user should be able to repetitively select and add products to an online cart that could be later checked out through a single submission. The payment aspect using the credit card should be implemented securely by integrating it into an existing payment gateway.

Note:

a. STRU already has a computerised inventory management system that links all the distributed outlets. This can be reused in the proposed online platform.
b. The solution should allow further improvements to include order delivery through a delivery force that would carry a PDA-based solution to accept credit cards and cash.

Exercise

1. Propose an architecture to realise the online platform in the SriToysRus Ltd (STRU) scenario given above.
    a. Identify the layers/tiers of the architecture.
    b. List the components of each layer/tier.
    c. Sketch / draw the overall architecture with clear labels for tiers and components.
2. List out and briefly describe the architectural patterns used in the proposed architecture.
3. Based on the description of the above scenario, identify a legacy system that needs to be integrated into the architecture and discuss a method for integrating it into the overall solution.
4. List high-level sub-systems of the proposed architecture.
5. Identify and describe an interaction/integration in the architecture where Asynchronous Communication is more suitable than Synchronous communication.

Answer 1

Proposed Architecture for the SriToysRus Online Platform

The proposed architecture for the SriToysRus (STRU) online platform will follow a multi-tiered approach, leveraging middleware to ensure scalability, reliability, and loose coupling between components. This architecture will be designed to integrate new services seamlessly, such as the future smartphone—based delivery system, without disrupting the existing structure.

1. Identification of Layers/Tiers

The architecture can be logically divided into four primary tiers:

- **Presentation Tier:** This is the user-facing layer. It is responsible for all interactions with the end user and presents the application's interface.
- **Business/Application Tier:** This is the core of the system. It handles the business logic, processing, and communication with other services. It acts as the "brain" of the application, orchestrating all the necessary operations. This is where the middleware will reside.
- **Integration Tier:** This tier is responsible for integrating with legacy and external services such as the Payment Gateway.
- **Data Tier:** This tier is responsible for all data storage and retrieval. It includes databases and external systems that provide or consume data.

2. Components of Each Layer/Tier

a. Presentation Tier

- **Web Application (Frontend):** The user interface (UI) for the online platform. This could be a Single-Page Application (SPA) built with a framework like React, Angular, or Vue.js, or a traditional server-rendered application. It handles user interactions such as:
    - Searching for toys.

    - Displaying product details.
    - Managing the shopping cart.
    - User registration and login.
    - Checkout process.
    - Displaying user profiles and order history.
- **Mobile Application (Optional/Future):** A native or cross-platform mobile app for iOS and Android, offering the same functionalities as the web application.

b. Business/Application Tier

This tier is crucial for decoupling the frontend from the data and external services. We can use a **N-Tier** or **Microservices Architecture** enabled by **Message-Oriented Middleware (MOM)** and **RESTful APIs**.

- **API Gateway:** A single entry point for all client requests. It handles routing requests to the appropriate service, authentication, and rate limiting (In the case of MSA).
- **User Service:** A service responsible for managing user profiles, authentication, and session management. It interacts with the Database.
- **Product & Inventory Service:** A service that acts as a wrapper around the existing computerised inventory management system. This service could be integrated to legacy inventory through an ESB or adaptor. It provides functionalities such as:
    - Fetching product details.
    - Checking real-time stock levels.
    - Updating stock after a sale.
    - Managing discounts.
- **Shopping Cart Service:** A service that manages the state of a user's shopping cart. It stores and retrieves items added to the cart.

- **Order Service:** A central service that orchestrates the checkout process. When a user submits an order, this service:
    - Receives the order details from the API Gateway.
    - Performs initial validation.
    - Sends a message to the Message Broker for asynchronous processing of payments and delivery.

c. Integration Tier (Middleware–Centric)

- **ESB–Payment Service:** An ESB that handles the secure integration with the existing **Payment Gateway**. It processes credit card details and confirms payment status. It communicates with the Order Service asynchronously via a Message Broker.
- **ESB–Delivery Service:** An ESB responsible for integrating with the **external Delivery Company Platform**. It receives delivery requests from the Message Broker and sends order information to the external system to initiate a delivery.
- **Message Broker (Pub–Sub–Middleware):** A central component (e.g., RabbitMQ or Apache Kafka) that facilitates asynchronous communication between the business logic services.
    - **Asynchronous Communication:** The Order Service sends a "New Order Placed" message to a message queue. The Payment Service and Delivery Service, which are both listening to this queue, can process the message independently and in parallel. This makes the checkout process faster and more resilient.
    - **Loose Coupling:** The business logic services don't need to know about each other's implementation details. They only need to know how to send and receive messages from the broker.
- **Existing Computerised Inventory Management System:** The current, centralised system that holds the product and stock data. The Product & Inventory Service will be a proxy to this system.

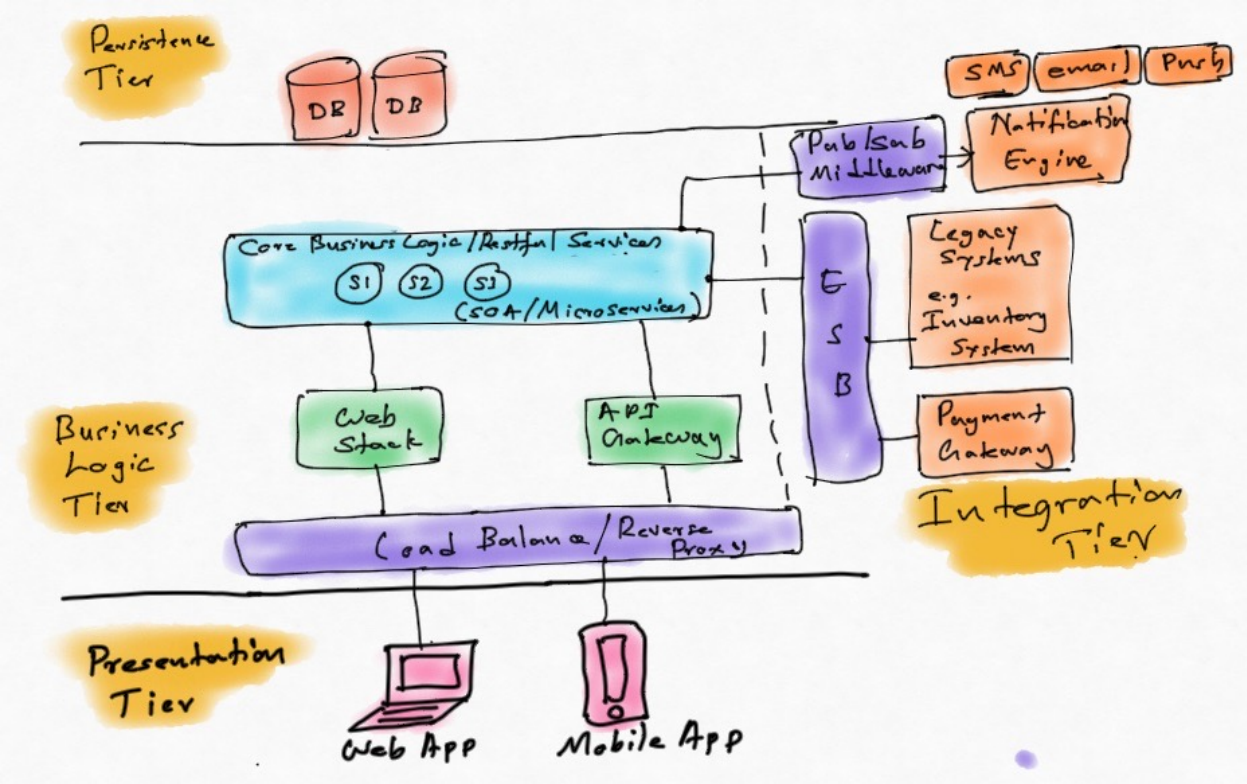
- **External Payment Gateway:** The third-party service that handles secure credit card processing.
- **External Delivery Company Platform:** The existing system used by the delivery company to track and monitor distribution.

d. Data Tier

- **User Database:** A database (e.g., MongoDB, PostgreSQL) to store user account information, profiles, wish lists, and past purchases.

3. Overall Architecture Sketch

- A multi-tiered architecture that augments a N-tier or Microservices architecture would be acceptable.



Answer 2

Based on the proposed architecture for the SriToysRus online platform, here are the architectural patterns used, with a brief description of how each is applied.

Architectural Patterns Used

1. **N or 3–Tier Architecture**

    - **Description:** This is a tiered architecture pattern that separates an application into three or more logical and physical tiers: a presentation tier (UI), an application/business tier (logic), and a data tier (database). Each tier is independent and can be developed, managed, and scaled separately. Communication flows typically from the presentation tier to the data tier, with the application tier acting as the intermediary.

    - **Application in the STRU Scenario:** The proposed architecture is fundamentally built upon this pattern.

        - **Presentation Tier:** Consists of the user-facing **Web Application** and the future **Mobile App**.

        - **Business/Application Tier:** The core logic is handled by the **API Gateway** and **SOA** or **Microservices** (User, Product & Inventory, Order, etc.). This tier also contains the middleware components, like the **ESB** and **Message Broker**.

        - **Data Tier:** Comprises the internal databases.

2. **SOA or Microservices Architecture**

    - **Description:** A SOA or microservices architecture is an approach to developing a single application as a suite of small services, each running in its process and communicating with lightweight mechanisms, often RESTful APIs or message

queues. These services are built around business capabilities and are independently deployable by fully automated deployment machinery.

- **Application in the STRU Scenario:** The Business/Application Tier is implemented using a services approach. Instead of one large monolithic application, the business logic is broken down into small, dedicated services for specific functions:
  - **User Service:** Manages user authentication and profiles.
  - **Product & Inventory Service:** Focuses solely on product and stock information.
  - **Order Service:** Orchestrates the complex checkout process.
  - **Payment Service:** Handles all payment-related logic and gateway integration.
  - **Delivery Service:** Manages the communication with the external delivery company. This design allows each service to be developed, scaled, and updated independently, which is ideal for a growing company like STRU.

3. Publish–Subscribe (Pub/Sub) Pattern

- **Description:** This is a messaging pattern where senders of messages (publishers) do not program the messages to be sent directly to specific receivers (subscribers). Instead, messages are categorised into classes or topics, and subscribers express interest in one or more of these topics. The middleware (message broker) handles the routing of messages from publishers to the appropriate subscribers. This pattern provides loose coupling, where the publisher and subscriber have no direct knowledge of each other.

- **Application in the STRU Scenario:** The **Message Broker** is the core component that implements this pattern.

- **Publisher:** The **Order Service** acts as the publisher. When a user successfully checks out, the Order Service publishes a message (e.g., "New Order Placed") to a specific topic or queue within the message broker.
- **Subscribers:** The **Payment Service** and the **Delivery Service** are subscribers. They are constantly "listening" for new messages on the "New Order Placed" topic. When a message arrives, they independently and asynchronously process it. The Payment Service will initiate the payment, and the Delivery Service will notify the delivery company. This pattern ensures that the checkout process is reliable and efficient. The Order Service doesn't have to wait for the Payment and Delivery services to respond synchronously, which could cause timeouts. If either of the subscribing services is temporarily unavailable, the message remains in the queue and will be processed once the service is back online, ensuring no data is lost.

It is optional to integrate the order management and payment gateway services through pub-sub middleware. These services could be integrated through the ESB in synchronous mode.

Answer 3

Based on the scenario description, the primary legacy system that requires integration is the **existing inventory management system**. The scenario states that this system "links all the distributed outlets" and can be "reused in the proposed online platform." This indicates it's an operational, non-internet-facing system that holds the authoritative data for product stock, which is a critical piece of the new solution.

Method to Integrate the Legacy System: The Service-Oriented Architecture (SOA) Approach

To integrate this legacy system, we will use a **Service-Oriented Architecture (SOA)** approach, specifically employing an **Enterprise Service Bus (ESB)** or an **Orchestrator** in conjunction with the **Adapter pattern**. This method provides robust, centralised control over the integration process.

1. The Integration Problem

Integrating a legacy system directly into a new, modern architecture presents challenges:

- **Incompatible Interfaces:** The legacy system may expose data or functions through outdated interfaces (e.g., SOAP, flat files, direct database access) that are not easily consumable by modern services.
- **Security and Performance:** The legacy system might lack the necessary security features and performance to handle a high volume of requests from an internet-facing application.
- **Tight Coupling:** Direct integration would tightly couple the new services to the old system's internal logic, making the entire solution brittle and difficult to maintain or evolve.

2. Proposed Solution: ESB/Orchestrator with an Adapter

To overcome these problems, the integration will be managed by a central ESB or Orchestrator. This component will act as the single point of contact for any service needing to interact with the inventory system.

A. The Adapter Pattern in an ESB/Orchestrator Context

The **Adapter** is a crucial component within the ESB/Orchestrator's integration flow. Its sole purpose is to abstract the complexities of the legacy system's interface.

- **Purpose:** The adapter acts as a translation layer. It exposes a standardised, modern interface (e.g., a RESTful API) to the rest of the new architecture while internally handling the specific and potentially complex communication with the legacy system.
- **Implementation:** The adapter would be implemented as a specific service or module within the ESB. It would contain the logic to:
    - **Convert modern requests** (e.g., a JSON message from the online platform) into the format the legacy system understands.
    - **Invoke the legacy function** (e.g., a stored procedure, a proprietary API call, or an older remote procedure call).
    - **Parse the legacy system's response** (e.g., a flat file, an XML message) and translate it back into a standardised format for the new services.

B. The Role of the ESB or Orchestrator

The ESB or Orchestrator is the central hub for this integration. Its key functions are:

- **Centralised Routing:** All requests related to inventory (e.g., "get stock level," "decrement stock") are routed through the ESB. The requesting service only needs to know the ESB's endpoint, not the specific details of the legacy system.

- **Orchestration and Mediation:** When a request to update stock comes in, the ESB can perform a sequence of actions:

    1. Receive the request from the **Order Service**.
    2. Route the request to the dedicated **Inventory Adapter** service.
    3. The adapter translates and forwards the request to the legacy system.
    4. The adapter receives a response and translates it back.
    5. The ESB then forwards the standardised response back to the **Order Service**.

- **Message Transformation:** The ESB can automatically handle data format conversions. For example, it could receive a JSON payload from a service and transform it into the XML or custom protocol required by the legacy system, all transparently to the requesting service.

- **Protocol Bridging:** If the legacy system uses a different communication protocol (e.g., a message queue, FTP), the ESB can bridge the gap, allowing the modern REST-based service to communicate with it seamlessly.

- **Monitoring and Reliability:** The ESB provides a central point to monitor all interactions with the legacy system. It can manage retries, handle errors gracefully, and prevent the online platform from failing if the legacy system is temporarily unavailable.

Example Integration Flow: Decrementing Stock

1. A user completes an online purchase via the **Web Application**.
2. The **Order Service** receives the completed order and needs to update the stock.
3. Instead of calling the legacy system directly, the **Order Service** sends a standardised request (e.g., a REST call POST /inventory/decrement_stock) to the **ESB/Orchestrator**.
4. The ESB receives the request and, based on its routing rules, directs it to the **Inventory Adapter** component.
5. The **Inventory Adapter** translates the request into the legacy system's native format (e.g., a function call with a specific set of parameters).

6. The adapter makes the call to the legacy **Inventory Management System**.
7. The legacy system processes the request and updates the stock level.
8. The legacy system returns a response to the adapter.
9. The adapter translates the legacy response into a standard format (e.g., a simple JSON response {"status": "success"}).
10. The ESB forwards this standardised response back to the **Order Service**, completing the transaction.

By using this approach, the legacy inventory system is successfully integrated into the modern architecture without exposing its internal complexities, ensuring the entire solution is robust, scalable, and easy to maintain.

Answer 4

Based on the proposed architecture for the SriToysRus online platform, here are the high-level sub-systems that make up the solution. These sub-systems are groupings of components that work together to perform a major function within the overall system.

1. **Presentation Sub-system:** This is the user-facing part of the application. Its primary responsibility is to present information to the user and capture their input.
    - **Components:** Web Application (the e-commerce website) and the future Mobile Application.
    - **Functionality:** Handles all user interactions, including browsing products, searching, user registration, managing the shopping cart, and the checkout process. It is the UI layer that the customer sees.
2. **ESB or API Gateway Sub-system:** This sub-system acts as the single entry point for all external traffic into the application's business logic. It is a critical part of the middleware.
    - **Components:** The API Gateway itself.
    - **Functionality:** Routes incoming requests from the presentation tier to the correct internal service, performs authentication and authorisation, and provides a layer of security by hiding the internal service structure from the outside world.
3. **Business Logic Sub-system:** This sub-system is the "brain" of the application, containing all the core business rules and logic. It is composed of multiple services that are independently deployable.
    - **Components:** User Service, Product & Inventory Service, Shopping Cart Service, Order Service, Payment Service, and Delivery Service.
    - **Functionality:** Each service handles a specific business capability, such as managing user profiles, checking stock levels, processing orders, and interact-

ing with payment and delivery platforms. This modularity allows for independent scaling and development.

4. **Pub-Sub/Middleware Sub-system:** This is the central communication layer that facilitates asynchronous, decoupled communication between the different services.
    - **Components:** The Message Broker (e.g., RabbitMQ, Apache Kafka).
    - **Functionality:** Manages message queues and topics. It ensures that critical events, such as a new order, are reliably broadcast to all interested services (e.g., Payment and Delivery) without the services needing to be directly aware of each other. Further, notifications such as email, SMS, and push could be managed through the pub-sub middleware.
5. **Data Management Sub-system:** This sub-system is responsible for the storage, retrieval, and management of all application data.
    - **Components:** Databases.
    - **Functionality:** Stores user information, maintains the single source of truth for data.

Answer 5

In the proposed architecture for the SriToysRus online platform, the most critical interactions where **asynchronous communication** is far more suitable than synchronous communication are the **notifications (email, SMS, app–push), order placement and fulfilment process**.

The Synchronous Alternative

If this interaction were designed synchronously, the flow would be as follows:

1. The user clicks "Checkout" on the web application.
2. The **Web Application** sends a request to the **Order Service** and waits for a response.
3. The **Order Service** calls the **Payment Service** and waits for the payment to be processed.
4. The **Payment Service** communicates with the **External Payment Gateway** and waits for a success or failure message.
5. After receiving a successful payment, the **Order Service** calls the **Delivery Service** and waits for the delivery to be scheduled with the **External Delivery Platform**.
6. The **Delivery Service** waits for the external platform to acknowledge the request.
7. Only after all these steps are completed does the **Order Service** send a success response back to the **Web Application**.

Problems with this synchronous approach:

- **High Latency:** The user would have to wait for all downstream systems to respond in sequence. If the payment gateway or the delivery platform is slow, the user's checkout experience would be frustratingly long, potentially leading to abandoned carts.
- **System Fragility:** The entire checkout process is a single point of failure. If the **Payment Service** or **Delivery Service** is temporarily down or unresponsive, the entire transaction fails, and the user receives an error. The user would have to start the process all over again.

- **Resource Inefficiency:** The server resources on the **Order Service** would be tied up waiting for responses from other services, which is an inefficient use of resources, especially under high load (e.g., during a sale).

The Asynchronous Solution

The proposed architecture solves these problems by using a **Message Broker** and the **Publish–Subscribe (Pub/Sub) pattern**.

Interaction Flow:

1. The user clicks "Checkout" on the web application.
2. The **Web Application** sends a request to the **Order Service**. This is a quick, synchronous call to validate the request.
3. The **Order Service** performs initial validation, creates a new order entry in its database with a "Pending" status, and immediately sends a success response back to the user. The user sees an "Order received, we are processing it" message. This is the critical moment of asynchronous decoupling.
4. The **Order Service** then publishes a "New Order Placed" message to the **Message Broker**. This is the key asynchronous step.
5. The **Payment Service** and the **Delivery Service**, which are both subscribers to the "New Order Placed" topic, independently receive and process this message.
    - The **Payment Service** processes the payment.
    - The **Delivery Service** schedules the delivery with the external platform.
6. Each service, upon completing its task, can send a message back to the broker to update the order status. The **Order Service** can listen for these messages and update the order from "Pending" to "Paid" and "Delivery Scheduled."

Benefits of this Asynchronous Approach:

- **Improved User Experience:** The user receives an immediate response after submitting their order, making the application feel fast and responsive. They don't have to wait for long-running processes to complete.
- **Resilience and Fault Tolerance:** The system becomes much more robust. If the **Delivery Service** is down for a few minutes, the "New Order Placed" message will simply wait in the message queue. When the service comes back online, it will automatically pick up the message and process it. No orders are lost, and the user's initial interaction was successful.
- **Decoupling:** The **Payment Service** and **Delivery Service** are completely independent. The **Order Service** doesn't need to know if they exist, their specific APIs, or their current state. It only needs to know how to publish a message to the broker. This allows each service to be developed, deployed, and scaled independently.
- **Scalability:** Under high traffic, the Message Broker can handle a large number of messages. You can simply add more instances of the **Payment Service** or **Delivery Service** to process messages faster, scaling only the components that are under heavy load.